



Dal Web ai Webservices

Prof. Davide Quaglia



HTTP/HTTPS

- Inventato da Tim Berners-Lee al CERN di Ginevra nel 1989
- Nato per la fruizione di contenuti in rete (World Wide Web)
- Oggi usato anche per l'invocazione di funzionalità remote →
Webservice

Esempio di HTTP/HTTPS

- Il client si chiama “web browser” (o semplicemente “browser”)
 - Firefox, Chrome, Edge, Safari, Opera, ...
- Il server si chiama “web server”
 - Apache, NGINX, NodeJS
- La comunicazione avviene sul protocollo TCP sulla rete Internet



Protocollo HTTP/HTTPS

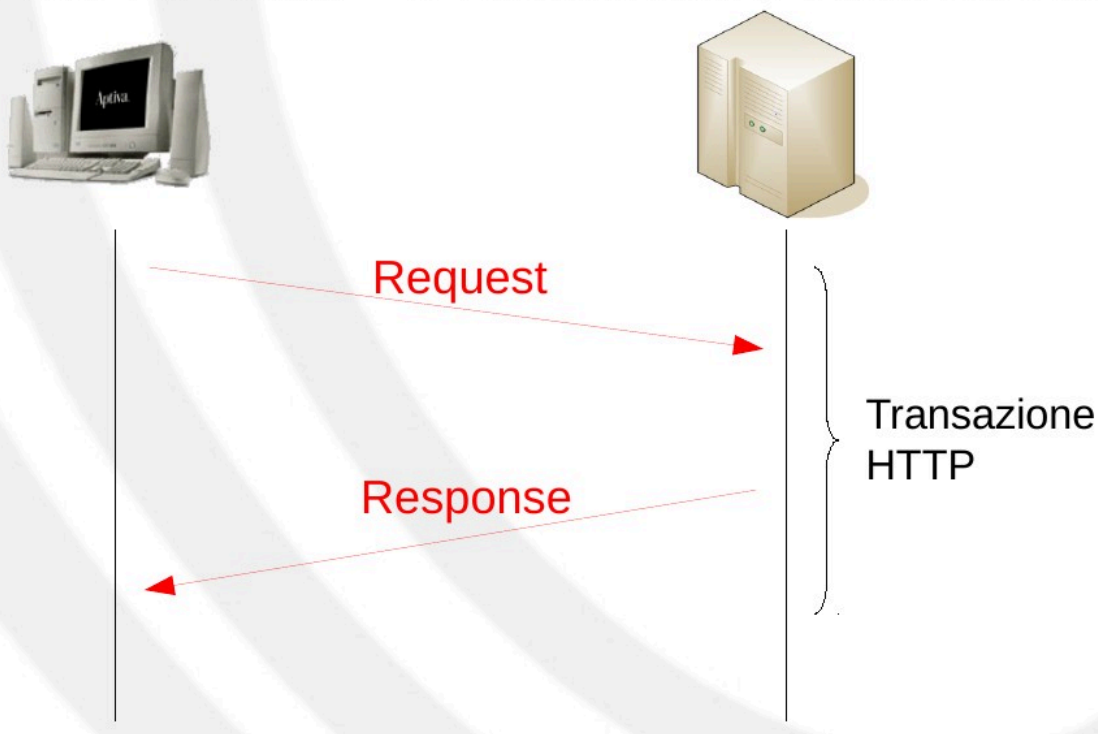
Browser Web (client)

IP: 157.27.12.5 Porta TCP: 3500

Server Web (server)

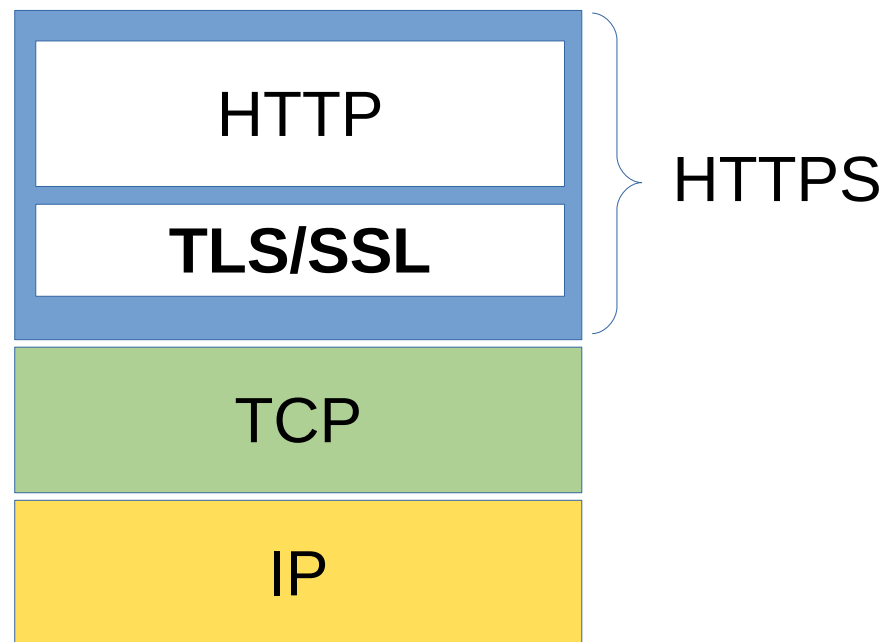
IP: 130.192.16.20 Porta TCP: 80

- Client/Server
- Testuale
- Fasi
 - 1) Apertura di una connessione TCP
 - 2) *[HTTPS] Autenticazione del server e negoziazione di una chiave di cifratura*
 - 3) Richiesta e risposta
 - 4) Chiusura della connessione TCP



HTTPS

- I messaggi che passano nella connessione TCP sono gli stessi dell'HTTP ma vengono sottoposti a
 - Cifratura dei dati in transito
 - Autenticazione del server mediante certificato digitale
- Il server lavora sulla porta 443 invece che 80



Esempio: richiesta

Metodo HTTP

URL

```
GET /it/i-nostri-servizi/servizi-per-studenti HTTP/1.1
User-Agent: Mozilla/5.0 (X11; U; Linux i686; en-US; rv:1.9.1.8)
Gecko/20100214 Ubuntu/9.10 (karmic) Firefox/3.5.8
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-us,en;q=0.5
Accept-Encoding: gzip,deflate
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7
Keep-Alive: 300
Connection: keep-alive
```

!!!Riga vuota!!!

Esempio: risposta

```
HTTP/1.1 200 OK
Date: Mon, 17 May 2022 16:10:48 GMT
Server: Apache
Last-Modified: Mon, 29 Mar 2022 13:57:17 GMT
Keep-Alive: timeout=15, max=100
Connection: Keep-Alive
Transfer-Encoding: chunked
Content-Type: text/html
```

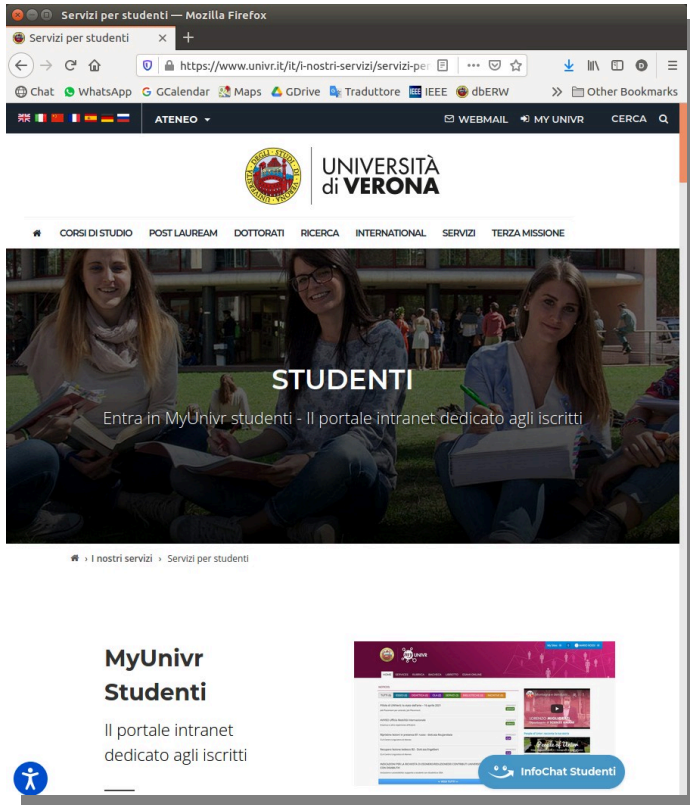
Intestazione della risposta

!!!Riga vuota!!!

```
<html>
...
</html>
```

Corpo della risposta

Esempio: visualizzazione della pagina



- Il corpo della risposta può contenere:
 - HTML puro
 - HTML + codice Javascript
 - Sequenza binaria che rappresenta un'immagine
 - Cascading Style Sheets (CSS)
 - Intere librerie di codice Javascript da eseguire sul browser

Hyper Text Markup Language (HTML)

- Linguaggio testuale di descrizione di una pagina
- Specializzazione del generico XML (eXtensible Markup Language)
- Basato su “tag” annidati eventualmente contenenti attributi

prova.html nella cartella Esempi-web/

```
<!-- My document -->
<HTML>
<HEAD>
  <TITLE>My Document</TITLE>
</HEAD>
<BODY>
  <H1>Header</H1>
  <P>
    Testo <b>grassetto</b>
  </P>
  <P>
    Testo <i>corsivo</i>
  </P>
</BODY>
</HTML>
```

Cascading Style Sheets (CSS)

css.html

```
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" href="styles.css">
</head>
<body>

<h1>This is a heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

styles.css

```
body {
  background-color: powderblue;
}
h1 {
  color: blue;
}
p {
  color: red;
}
```

Cascading Style Sheets (CSS)

css.html

```
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" href="styles.css">
</head>
<body>

<h1>This is a heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

styles.css

```
body {
  background-color: powderblue;
}
h1 {
  color: blue;
}
p {
  color: red;
}
```

Provate a togliere questo tag oppure a cambiare il contenuto di styles.css

HTML: tag per richiamare immagini

immagini.html

```
<!DOCTYPE html>

<html>

<body>


<h2>I trulli di Alberobello</h2>




</body>

</html>
```

HTML: tag per richiamare immagini

immagini.html

```
<!DOCTYPE html>
<html>
<body>

<h2>I trulli di Alberobello</h2>


</body>
</html>
```

Vedi
Content Delivery Network

Provate a sostituire questa URL con
<https://cdn.britannica.com/70/20070-050-C2E2045C/Central-Park-Manhattan-New-York-City-apartment.jpg>

HTML: tag per il collegamento ipertestuale

link.html

```
<!DOCTYPE html>
<html>
<body>

<h1>HTML Links</h1>

<p><a href="https://www.w3schools.com/">Visit W3Schools.com!</a></p>

</body>
</html>
```

HTML: tag per il collegamento ipertestuale

link.html

```
<!DOCTYPE html>
<html>
<body>

<h1>HTML Links</h1>

<p><a href="https://www.w3schools.com/">Visit W3Schools.com!</a></p>

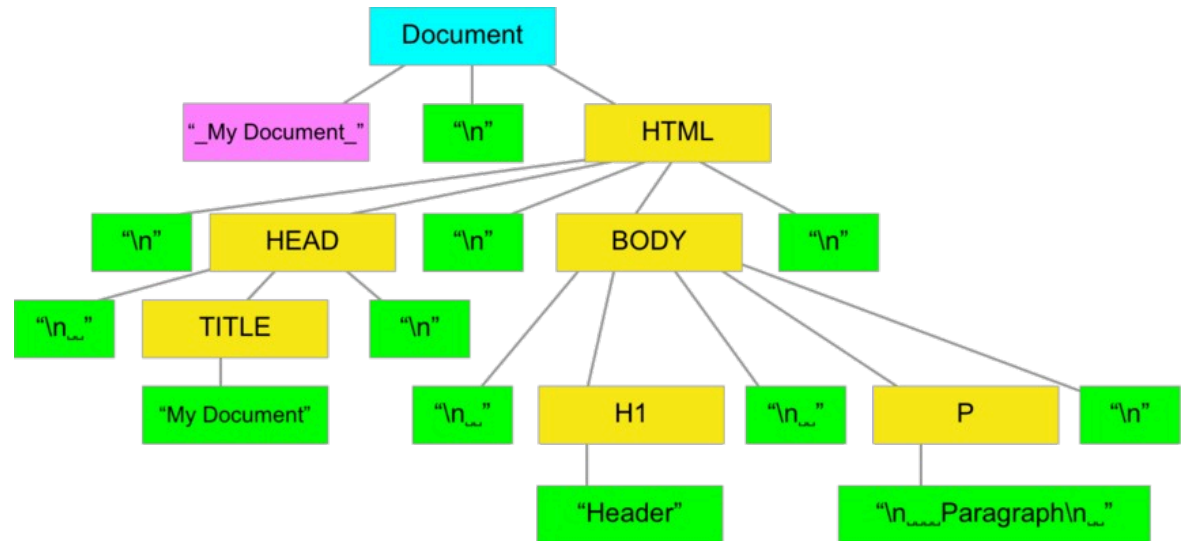
</body>
</html>
```

Provate a far diventare il link di colore rosso!

Document Object Model (DOM)

- rappresentazione della pagina HTML come modello orientato agli oggetti
- standard ufficiale del W3C

```
<!-- My document -->
<HTML>
<HEAD>
  <TITLE>My Document</TITLE>
</HEAD>
<BODY>
  <H1>Header</H1>
  <P>
    Paragraph
  </P>
</BODY>
</HTML>
```



Javascript

javascript.html

```
<!DOCTYPE html>
<html>
<body>

<h2>Use JavaScript to Change Text</h2>
<p>This example writes "Hello JavaScript!" into an HTML element with id="demo":</p>

<p id="demo"></p>

<script>
    document.getElementById("demo").innerHTML = "Hello JavaScript!";
</script>

</body>
</html>
```

Javascript: cosa e dove?

- Codice simil-Java
 - direttamente contenuto nelle pagine HTML attraverso il tag <script>

```
<script>  
document.getElementById("demo").innerHTML = "Hello JavaScript!";  
</script>
```

- interi file di codice (librerie) richiamati nell'HTML e poi scaricati dal browser

```
<script src="myscripts.js"></script>
```

Javascript ed eventi

- Il codice Javascript può essere associato a degli eventi che l'utente genera interagendo con la pagina
 - Pressione di un bottone
 - Selezione di una casella
 - ...

Javascript ed eventi: esempio

javascript-evento.html

```
<!DOCTYPE html>
<html>
<body>

<button
onclick="document.getElementById('demo').innerHTML=Date()">
Che ora è?
</button>

<p id="demo"></p>

</body>
</html>
```

Javascript ed eventi: esempio

javascript-evento.html

```
<!DOCTYPE html>
<html>
<body>

<button
onclick="document.getElementById('demo').innerHTML=Date()">
Che ora è?
</button>

<p id="demo">
</p>
</body>
</html>
```

Provate a cliccare più volte sul bottone.
Cosa succede?

Javascript e Document Object Model (DOM)

- Il DOM trasforma una pagina web da documento statico a Graphical User Interface (GUI)
- il codice Javascript contenuto nella pagina HTML ed eseguito dal browser può agire sull'oggetto che rappresenta la pagina e modificarla → una pagina si automodifica come reazione a certe azioni scatenate dall'utente → comportamento simile ad una applicazione → web application
- Programmazione “lato client” (o “client side” o “lato front-end”)
- Alternative a Javascript
 - Typescript

Javascript e DOM

javascript-load.html

```
<!DOCTYPE html>
<html>
<body>

<iframe id="area" height="2000" width="1000"></iframe>


<script>
  document.getElementById("area").src =
  "https://www.ansa.it/sito/notizie/topnews/index.shtml";
</script>

</body>
</html>
```

Javascript e gestione del tempo

javascript-timer.html

```
<!DOCTYPE html>
<html>
<body>

<h1>The Window Object</h1>
<h2>The setInterval() Method</h2>

<p id="demo"></p>

<script>
setInterval(displayHello, 1000);

function displayHello() {
    document.getElementById("demo").innerHTML += "Hello";
}
</script>

</body>
</html>
```

Javascript e gestione del tempo

javascript-timer2.html

```
<!DOCTYPE html>
<html>
<body>

<h1>The Window Object</h1>
<h2>The setInterval() Method</h2>

<p id="demo"></p>

<script>
setInterval(function() {document.getElementById("demo").innerHTML += "Hello"}, 1000);
</script>

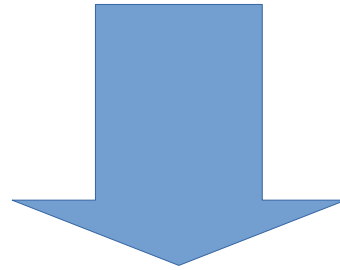
</body>
</html>
```

Esercizio

- Scrivere e provare una pagina HTML che ricarica periodicamente il sito dell'ANSA

Esercizio

- Scrivere e provare una pagina HTML che ricarica periodicamente il sito dell'ANSA



APPROCCIO **SINCRONO** AL REFRESH DEI CONTENUTI

- in gergo informatico si dice “polling”
- non è detto che quel periodo di refresh sia quello giusto
 - refresh inutili con spreco di risorse di rete e calcolo
 - mancato refresh quando davvero serve
- l'alternativa è un approccio **ASINCRONO** o **EVENT-DRIVEN** (vedi programmazione mediante Web Socket)

Uniform Resource Locator (URL)

- Detto anche Universal Resource Locator
- Stringa che permette di identificare in maniera univoca una risorsa HTTP in qualsiasi parte della rete mondiale
- Struttura
 - Protocollo utilizzato a livello applicazione → protocollo di livello trasporto + porta utilizzata (HTTP → TCP/80, HTTPS → TCP/443)
 - Nome dell'host (o indirizzo IP) che eroga tale risorsa
 - Nome della risorsa con suo **percorso logico** completo (non necessariamente fisico)
- La porta può essere indicata esplicitamente se non è quella standard

<https://www.univr.it/servizi/studenti/carriera>

<https://www.univr.it:8000/servizi/studenti/carriera>

Un semplice web server

- Aprire il file `serverHTTP.c` in `Esempi-web/` e analizzarne il contenuto
- Compilarlo come nell'esercitazione sull'interfaccia socket ed eseguirlo
- Provare ad aprire un secondo terminale nella stessa cartella e a rilanciare lo stesso server. Funziona? Perché?
- Aprire il browser preferito e impostare la URL
“`http://127.0.0.1:8000/`” Cosa si vede sul browser e sul terminale?
- Aprire il browser preferito e impostare la URL
“`http://localhost:8000/`” Cosa cambia?

form-get.html

Passare dei dati al server web col metodo GET

```
<!DOCTYPE html>
<html>
<body>

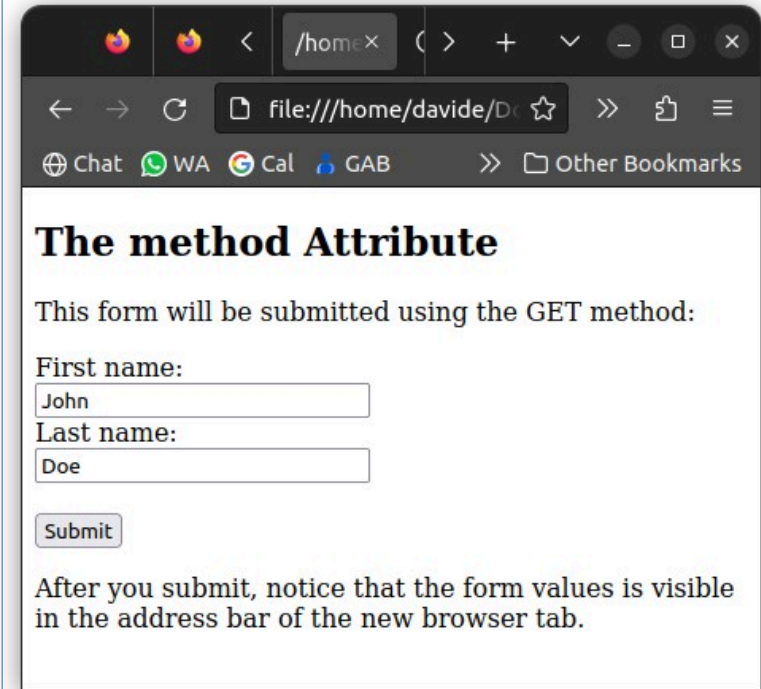
<h2>The method Attribute</h2>

<p>This form will be submitted using the GET method:</p>

<form action="/action" target="_blank" method="get">
  <label for="fname">First name:</label><br>
  <input type="text" id="fname" name="fname" value="John"><br>
  <label for="lname">Last name:</label><br>
  <input type="text" id="lname" name="lname" value="Doe"><br><br>
  <input type="submit" value="Submit">
</form>

<p>After you submit, notice that the form values is visible in the
address bar of the new browser tab.</p>

</body>
</html>
```



Esercizio

- Eseguire il server web `serverHTTP.c` e con il browser preferito aprire il file `form-get.html`
- Cosa si vede?
- Provare ad analizzare il contenuto della connessione TCP con Wireshark.
- Cosa si vede?

E corrispondente richiesta HTTP

► Parametri (max 2048 caratteri)

```
GET /action?fname=John&lname=Doe HTTP/1.1
User-Agent: Mozilla/5.0 (X11; U; Linux i686; en-US; rv:1.9.1.8)
Gecko/20100214 Ubuntu/9.10 (karmic) Firefox/3.5.8
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-us,en;q=0.5
Accept-Encoding: gzip,deflate
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7
Keep-Alive: 300
Connection: keep-alive
```

!!!Riga vuota!!!

form-post.html

Passare dei dati al server web col metodo POST

```
<!DOCTYPE html>
<html>
<body>

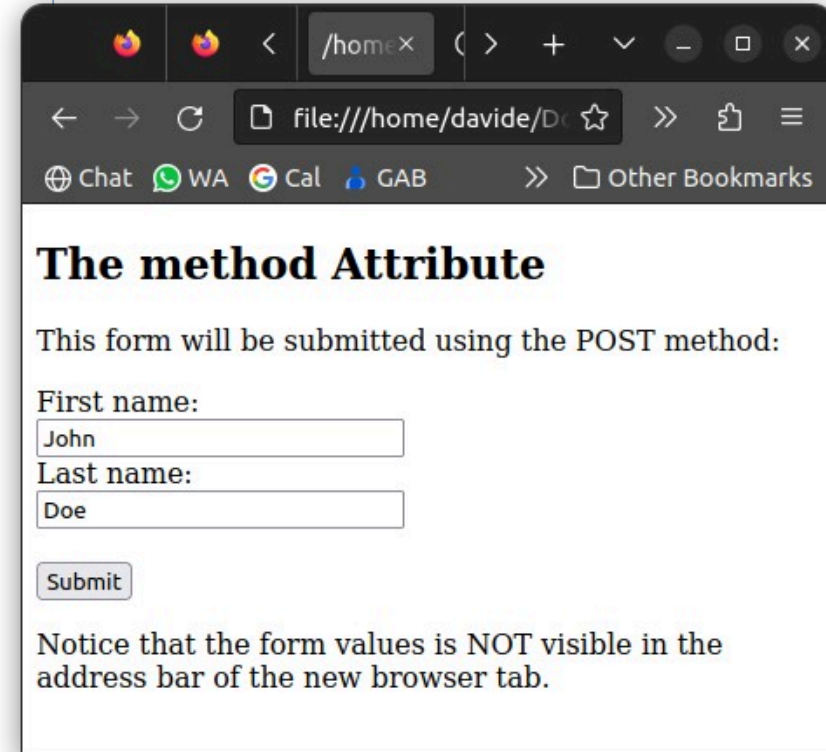
<h2>The method Attribute</h2>

<p>This form will be submitted using the GET method:</p>

<form action="/action" target="_blank" method="post">
  <label for="fname">First name:</label><br>
  <input type="text" id="fname" name="fname" value="John"><br>
  <label for="lname">Last name:</label><br>
  <input type="text" id="lname" name="lname" value="Doe"><br><br>
  <input type="submit" value="Submit">
</form>

<p>After you submit, notice that the form values is visible in the
address bar of the new browser tab.</p>

</body>
</html>
```



Esercizio

- Eseguire il server web `serverHTTP.c` e con il browser preferito aprire il file `form-post.html`
- Cosa si vede?
- Provare ad analizzare il contenuto della connessione TCP con Wireshark.
- Cosa si vede?

E corrispondente richiesta HTTP

POST /action HTTP/1.1

User-Agent: Mozilla/5.0

Accept: text/html

Accept-Language: en-us,en;q=0.5

Accept-Encoding: gzip,deflate

Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7

Keep-Alive: 300

Connection: keep-alive

!!!Riga vuota!!!

fname=John&lname=Doe

!!!Riga vuota!!!

Intestazione della richiesta

Corpo della richiesta

NESSUN LIMITE
DI CARATTERI !

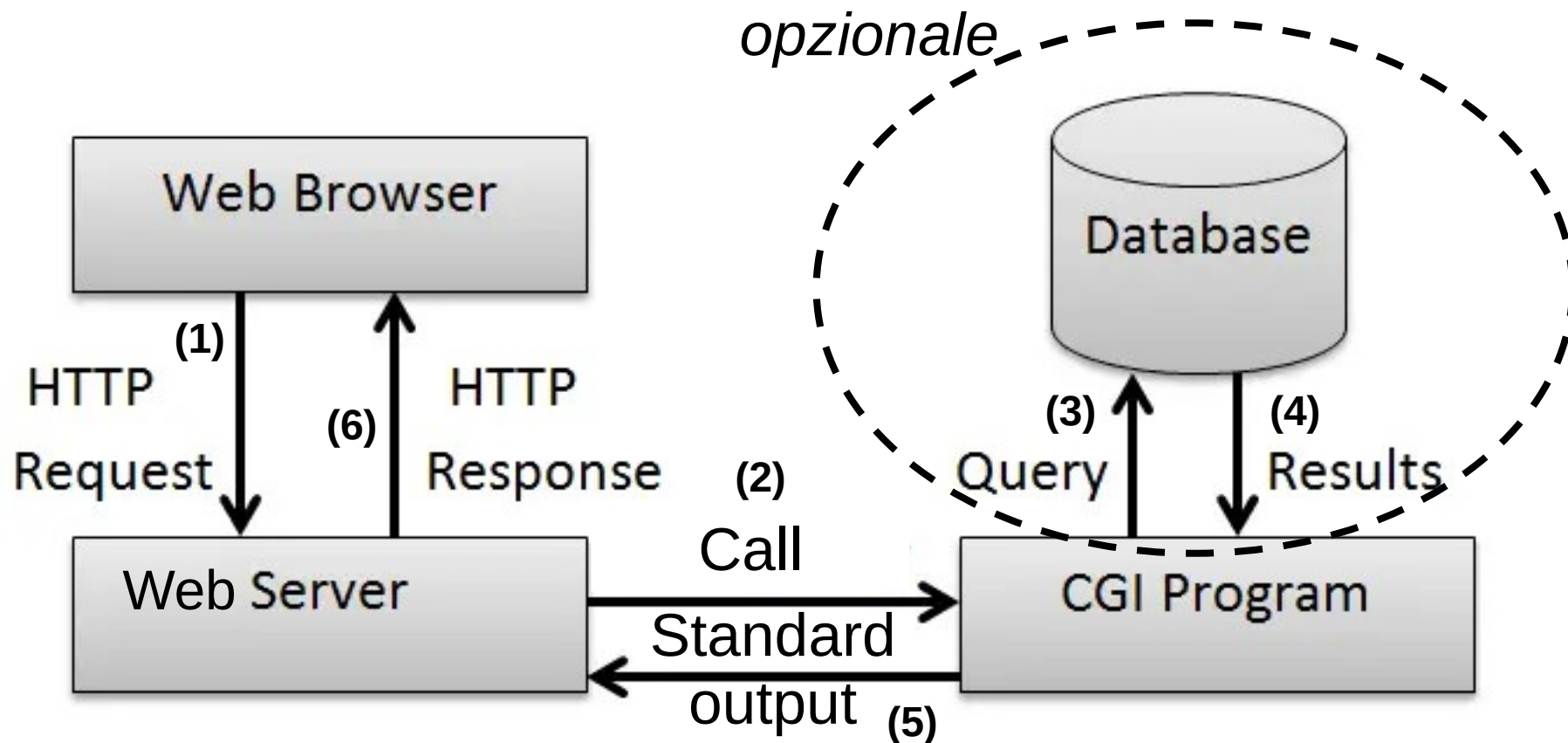
Esercizio

- Modificare il file `serverHTTP.c` in modo che, invece di restituire sempre la solita pagina web di prova, restituisca una delle pagine html usate dal lucido 9 in poi scelta attraverso l'uso del browser.
- Suggerimenti:
 - Provare a fare la nuova richiesta col browser usando il `serverHTTP.c` in modo da vedere la richiesta HTTP per capire dove si trova la stringa con il nome del file nella richiesta che fa il browser (aiutarsi anche con Wireshark)
 - Cosa devo scrivere nella barra del browser?
 - Capire come recuperare la stringa con il nome del file dalla richiesta HTTP (si veda `esempio-parser.c`)
 - Per costruire la risposta riciclare parte del codice usato nell'esercizio del trasferimento di file
- Prova finale: cosa succede se chiedo al server di restituire i file `form-get.html` e `form-post.html` ?

Esercizio per casa

- Modificare il server web `serverHTTP.c` in modo che accetti con il metodo POST un intero file da salvare nella cartella del server (il cosiddetto “upload sul server”).
- Suggerimenti:
 - utilizzare il file `form-file.html` con `serverHTTP.c` non modificato ed analizzare il contenuto della connessione TCP con Wireshark in modo da capire nella richiesta HTTP
 - Dove si trova il nome del file
 - Dove si trova il contenuto del file
 - Nella lettura della richiesta HTTP sul server aggiungere il codice che salva il file prendendo spunto dal codice usato nell’esercizio del trasferimento di file
 - Invece la risposta HTTP può essere molto statica come nella versione originale di `serverHTTP.c`

Common Gateway Interface (CGI)



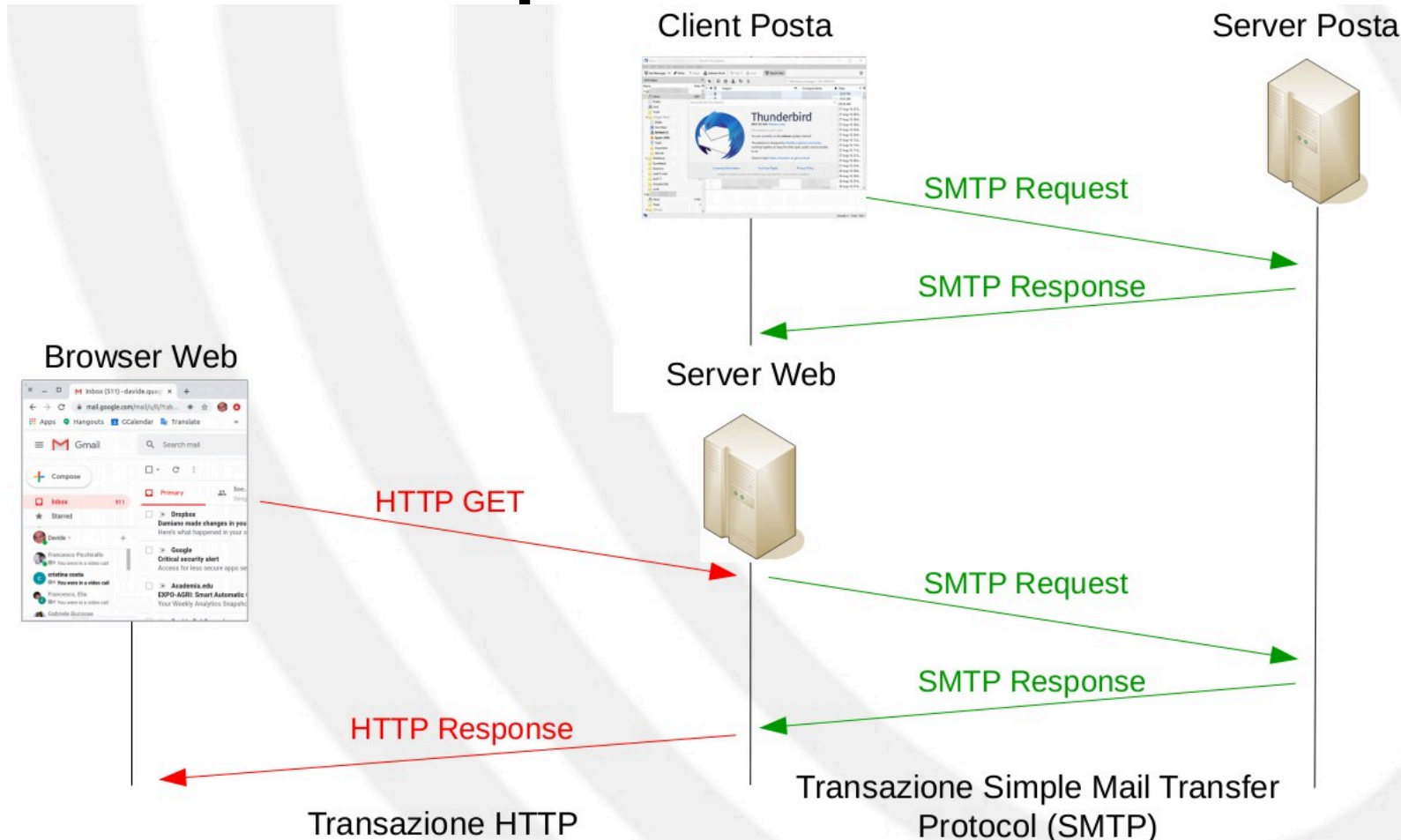
Esempio di web server esteso con gestione CGI

- Aprire il file `serverHTTP-CGI.c` in `Esempi-web/` e analizzarne il contenuto
- Compilarlo come al solito ed eseguirlo
- Aprire il browser preferito e il file `sommatrice-web.html`
 - Cosa si vede sul browser?
 - NOTA: Provare con numeri positivi, negativi, con parte decimale...
- A cosa corrisponde il secondo parametro della funzione `sommatrice()` ?

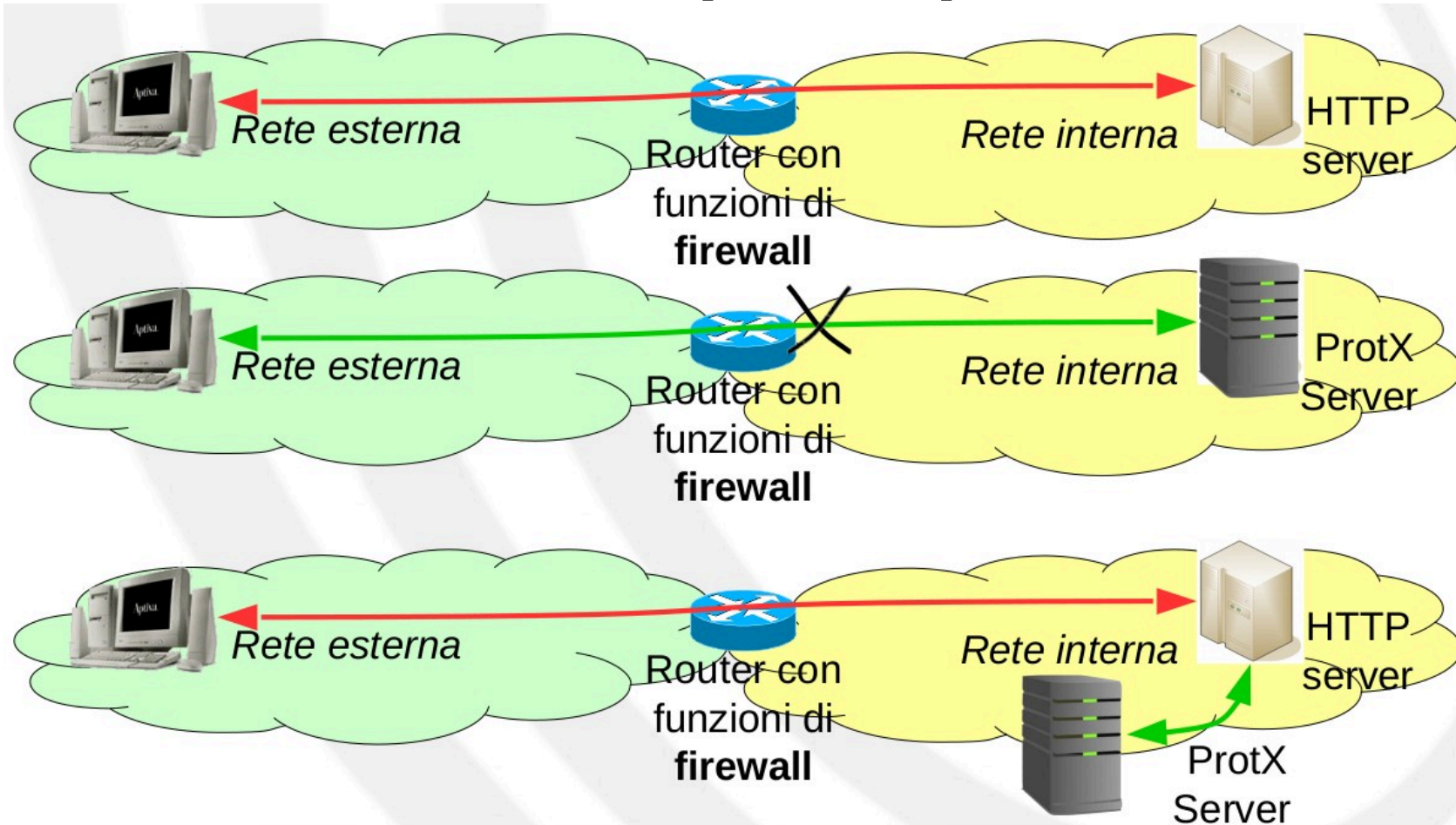
CGI e web dinamico

- Il risultato della richiesta del browser non è più un doc statico
- Il programma CGI viene eseguito “lato server” (o “server side” o “lato back-end”)
 - Eseguitibile (ad esempio client di posta elettronica)
 - PHP
 - Perl
 - Java, JSP, Servlet
 - Active Server Page (ASP)
 - NodeJS
 - Fastify
 - ... qualsiasi cosa la moda si inventerà ...
- Il browser web diventa il client di molte applicazioni di rete:
 - Posta elettronica
 - Wiki
 - Content Management Systems (CMS)

Web e posta elettronica

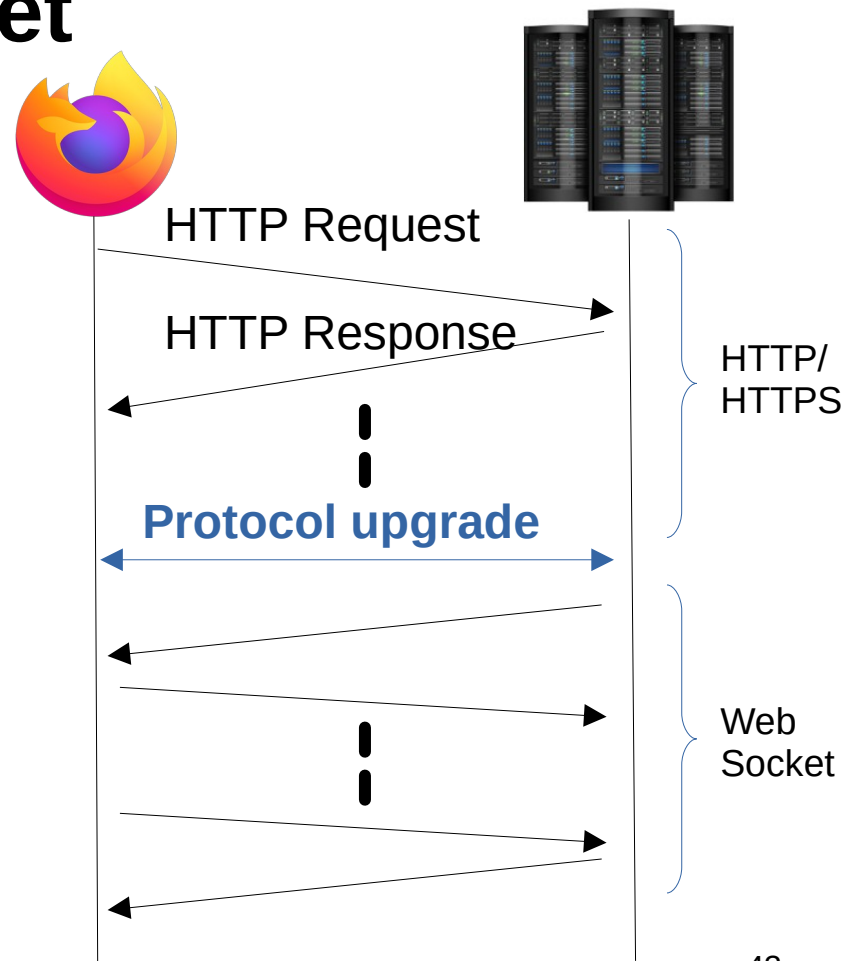


Perché tutto passa per il web?



Web socket

- Protocollo di livello applicazione
ALTERNATIVO a HTTP/HTTPS
- Ammette la comunicazione SIMMETRICA tra
il web browser e il web server
- Nasce da una sessione HTTP/HTTPS
attraverso una operazione di “Protocol
upgrade”
- Da quel momento in poi entrambi i processi
possono “prendere l’iniziativa” per mandare
dei dati dall’altra parte
 - Utilizzato per il REFRESH ASINCRONO di una
pagina web attraverso il PUSH di una nuova
pagina da parte del server





Protocol upgrade

GET / HTTP/1.1

Host: server.example.com

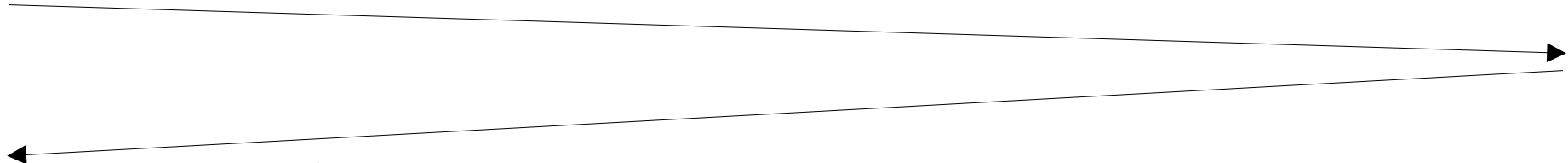
Upgrade: websocket

Connection: Upgrade

Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==

Origin: http://example.com

Sec-WebSocket-Version: 13



HTTP/1.1 101 Switching Protocols

Upgrade: websocket

Connection: Upgrade

Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=

Esercizio su Web Socket

- Realizzazione di una chat eseguita dentro un browser web
 - Fare riferimento al materiale apposito condiviso con gli studenti

